

SENIOR FULL-STACK DEVELOPER · 7+ YEARS

I build applications businesses **rely on.**

Real-time web applications — the kind where state has to stay correct across many users at once. Operations platforms, browser-based telephony and healthcare portals, shipped to web, mobile and desktop.



7+

Years of experience

100+

Features & modules shipped

200+

Users on a live platform

EMAIL

gireesh.rajput786@gmail.com

LINKEDIN

[/in/gireesh-singh-293b83147](https://www.linkedin.com/in/gireesh-singh-293b83147)

PORTFOLIO

gireeshsingh.vercel.app

ENGAGEMENT

Remote · Contract / Full-time

React

Next.js

TypeScript

Node.js

NestJS

PostgreSQL

WebSockets

React Native

Electron

Seven years of production front-end, end-to-end.

I build real-time web applications — the kind where state has to stay correct across many users at once. Recent work includes an operations platform running 100+ trucks across multiple sites, a browser-based phone system handling conference calls and live transfers, and a healthcare portal where clinicians call patients while their health data streams onto the same screen. Seven years across React, Next.js and TypeScript on the frontend, and Node.js, NestJS and PostgreSQL on the backend — shipped to web, mobile and desktop.

7+

Years building production software

100+

Features & modules shipped

200+

Users on a single live platform

WHAT CLIENTS COME TO ME WITH

- An operational workflow that has outgrown spreadsheets and paper
- A dashboard that has to stay correct while several people edit at once
- A React codebase that gets slower to change with every feature
- A product that now needs a mobile or desktop presence, not three codebases
- Live data — calls, tickets, jobs — drifting out of sync between clients
- State bugs nobody can reproduce

HOW I WORK WITH CLIENTS

- **Fully remote**, with working hours arranged to overlap US, UK and Gulf business days
- **Clear written communication** — architecture and trade-offs documented before code
- **Agile delivery**, with review-ready increments rather than one large drop
- **Contract, retainer or full-time** engagements
- **Integrates with your team** — follows the conventions of the existing codebase
- **Handover-ready** — typed, modular code the next developer can follow

The thread through every project

Each platform in this portfolio has the same hard centre: many people acting on the same live data at the same time, each with a different role and a different view. Getting that right — a state model where illegal combinations simply cannot be represented — is the work I specialise in.

What I build

Seven areas of work, and the problems that usually bring a client to each one.

SaaS Applications

Multi-role SaaS platforms, dashboards, subscription products and internal business tools built to survive real operational use.

SOLVES

- Multiple user roles needing different views of the same data
- Operational workflows that outgrew spreadsheets
- Dashboards that must stay correct under concurrent edits

Real-Time Web Applications

Live dashboards, notifications and synchronised state over WebSockets — where being stale by ten seconds is a business problem.

SOLVES

- Users refreshing to find out what changed
- Two people acting on the same record at once
- Call, ticket or job state drifting between clients

React & Next.js Applications

Production React and Next.js builds with TypeScript, sane state ownership and an architecture the next developer can follow.

SOLVES

- A codebase that slows down with every feature
- Server and client state tangled together
- No clear boundary between UI and business rules

Mobile & Desktop Applications

The same product on every surface it needs to reach — responsive web, React Native on mobile and Electron on desktop, sharing logic rather than being rebuilt three times.

SOLVES

- A web product that needs a real desktop or mobile presence
- Three codebases drifting apart with every release
- Desktop capabilities the browser cannot reach

Business Platforms

Workflow-heavy applications with multiple roles, permissions and operational processes that mirror how the business actually runs.

SOLVES

- Paper or manual processes that need digitising
- Permission rules scattered through the UI
- Multi-site or multi-team coordination

Complex Frontend Systems

Large applications with heavy state, live data, many integrations and workflows that cannot be modelled as simple CRUD.

SOLVES

- State bugs nobody can reproduce
- Slow, janky interfaces under real data volumes
- Integration logic leaking into components

AI-Powered Applications

AI-assisted workflows and chat interfaces, with model output treated as untrusted input — validated before it reaches business rules. Solves: AI features that break on unexpected model output · chat interfaces needing streaming and conversation state · deciding where AI genuinely beats a deterministic solution.

MyTruckBoss

A digital ticketing and operations platform for dump-truck and sitework contractors, replacing paper ticket tracking with a centralised workflow for trucks, loads, tickets, drivers, payments and site operations across multiple construction sites.

Next.js

TypeScript

Redux Toolkit

TanStack Query

WebSockets

100+

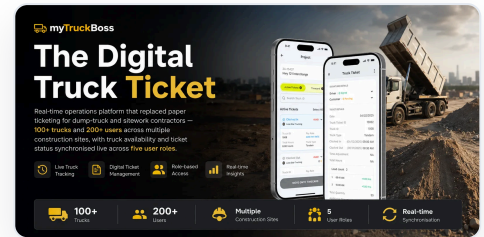
TRUCKS MANAGED

200+

PLATFORM USERS

5

USER ROLES



The challenge

Contractors tracked trucks by load or by hour on paper. Coordinating multiple sites took significant manpower, and no one had a shared view of what was happening on the ground.

- Managers could not see which trucks were available or already assigned
- Material requirements and delivered quantities were tracked manually
- Pending ticket status was invisible until paperwork caught up
- Accounting had no timely visibility into tickets or payments
- Teams fell out of sync because information was never centralised

My role — Senior Frontend Developer

I designed and implemented the frontend application and its supporting data architecture, working against a separately built Python/Django backend.

- Frontend architecture and reusable component system
- Redux state architecture for complex operational workflows
- TanStack Query data layer and backend API integration
- WebSocket-based real-time synchronisation
- Dashboards, operational views and analytics
- Role-specific experiences for five distinct user types

Real-time synchronisation & state architecture

When a truck becomes available, that state propagates to every connected user immediately — a manager sees it and can assign it to a load without refreshing. Ticket updates flow the same way. Admin, Manager, Truck Operator, Site Supervisor and accounting users each need a different slice of the same live data, so the component and state architecture treats role as a first-class input rather than bolting on per-role conditionals. Server state runs through TanStack Query, client and session state through Redux Toolkit, with a clear boundary between the two.

FRONTEND

Next.js

TypeScript

Redux Toolkit

TanStack Query

Axios

WebSockets

BACKEND (SEPARATE TEAM)

Python

Django

OUTCOME •

Digitised a workflow that ran on paper tickets and manual coordination. Centralising truck, ticket, material, driver and payment data gave construction teams a shared real-time view and cut the effort of keeping multiple sites in sync.

IHS Phone

A unified communication platform for businesses, call centres, insurance organisations and private nursing companies — internet-based calling alongside messaging, fax, voicemail and call management, letting an organisation run one business number with many extensions so employees share a number while keeping their own.

React

TypeScript

SIP.js

FreeSWITCH

WebSockets



The challenge

Traditional phone systems get unmanageable when many employees share a business number but each need their own extension and call workflow. The application had to deliver a modern web experience while supporting professional telephony behaviour end to end.

- Conference calls with multiple participants
- Blind transfers — hand a live call straight to another extension
- Attended transfers — speak to the receiving extension first
- Multiple simultaneous calls, placed or received while one is active
- Call hold across all of the above
- Messaging, fax, voicemail and real-time call history

My role — Frontend, functionality & real-time

I built the frontend application, the application-side functionality and the real-time synchronisation layer. The backend was implemented separately.

- Frontend architecture and reusable UI component system
- Complete application interface implementation
- API integration and application functionality
- WebSocket-based real-time synchronisation
- Managing complex communication state
- Call management, messaging, fax and voicemail interfaces
- Call history, dashboards and analytics

Call state management — the hard part

Supporting real telephony rather than basic one-to-one calling was the core difficulty. A user can join several participants into a conference, transfer a live call blind or attended, place a caller on hold to take another, and move between simultaneous calls — each of which mutates call state in a different way. Active calls, conferences, transfers, holds and inbound real-time updates all had to stay coherent in one interface, so the state model was built to make illegal combinations unrepresentable rather than defending against them case by case.

FRONTEND

React

Vite

TypeScript

Redux Toolkit

TanStack Query

WebSockets

COMMUNICATIONS

SIP.js

FreeSWITCH

Bandwidth telephony

OUTCOME •

Gave organisations a centralised software phone system in place of physical handsets — multiple extensions under a shared business number, with calling, messaging, fax, voicemail and history in one unified workflow.

Remote Patient Monitoring

A platform for clinicians monitoring patients remotely, combining live health data, in-portal voice calls, clinical messaging and documentation, and the billing and payroll workflows that nursing staff depend on.

React

TypeScript

SIP.js

WebRTC

REST API

The challenge

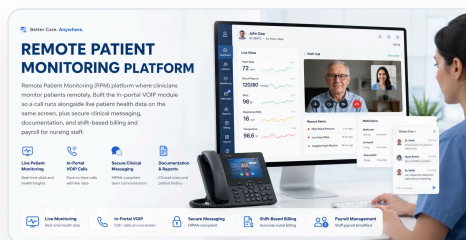
Clinical staff were switching between a phone, a monitoring dashboard and a notes system during a single patient interaction. Context was lost at every switch, and healthcare data carries constraints ordinary business software does not.

- Calling a patient meant leaving the screen showing their vitals
- Clinical notes and messages lived apart from the monitoring data
- Nursing compensation is shift-based and was tracked separately
- Patient data demands strict handling and access control

My role — Senior Frontend Developer

I built the in-portal VOIP module and the surrounding clinical workflows, integrating with a separately built backend.

- In-portal VOIP calling alongside live patient data
- Secure clinical messaging interfaces
- Clinical documentation and patient history views
- Shift-based billing and payroll workflows for nursing staff
- Real-time health data presentation
- API integration and application state



Calls and clinical data on one screen

The core of the work: a clinician starts a call from inside the portal and the patient's vitals, alerts and medication list stay visible throughout — no window switching during a clinical conversation, which is where context gets lost. Messaging, documentation and patient history sit on the same surface as monitoring, so what a clinician observes and what they record do not live in separate systems. Nursing staff are paid by shift, so billing and payroll were built into the platform rather than handled outside it.

FRONTEND

React

TypeScript

SIP.js

WebRTC

REST APIs

BACKEND (SEPARATE TEAM)

Python

Django

OUTCOME •

Gave clinicians a single place to monitor, call, message and document, and gave the organisation shift-based billing and payroll in the same system.

Client Web Application Suite

Production applications delivered across two agencies with React and Next.js on Agile cycles, with full cross-browser support. Included API integration work that cut data-sync response time by 40%, and peer code review that reduced bug resolution time by 25%.

Private client work — architecture happy to be discussed on a call.

Next.js

React

TypeScript

SASS

Agile

40%

Faster data-sync response after API integration work

25%

Reduction in bug resolution time via peer code review

2

Agencies, delivering on Agile cycles

ENGINEERING CAPABILITIES

Proven in production, not just on a CV

Frontend Architecture

Maintainable React and Next.js structure: reusable components, clear state ownership, and boundaries that keep business rules out of the view layer.

Proven in: [MyTruckBoss](#) · [IHS Phone](#)

State Management

Redux Toolkit for client and session state, TanStack Query for server state, and an explicit boundary between them — the distinction most large React codebases get wrong.

Proven in: [MyTruckBoss](#)

Real-Time Systems

WebSocket synchronisation, live updates and shared state across many concurrent users, including telephony call state that has to stay coherent through transfers, holds and conferences.

Proven in: [IHS Phone](#) · [MyTruckBoss](#)

API Integration

Reliable API communication: validation, error states, loading and empty states, caching, and integration logic kept out of components.

Proven in: [MyTruckBoss](#) · [Remote Patient Monitoring](#)

Complex Business Workflows

Turning operational processes into role-aware application flows — five distinct roles reading one live dataset, each with a different job to do.

Proven in: [MyTruckBoss](#) · [Remote Patient Monitoring](#)

Cross-Platform Delivery

Shipping one product to web, mobile and desktop — React Native for native mobile and Electron for desktop, sharing domain logic and component patterns instead of maintaining three separate builds.

From desktop and mobile work outside the case studies here

Production Engineering

Security, failure handling, accessibility and maintainability treated as part of the feature rather than a later pass.

Proven in: [IHS Phone](#) · [Remote Patient Monitoring](#)

AI Integration Patterns

AI-assisted workflows and chat interfaces built with streaming and conversation state, and model output validated as untrusted input before it reaches business rules.

[OpenAI API](#) · [LLM integration patterns](#)

PROCESS

How I work

The same five steps on every engagement, whether it is a two-week module or a platform build.

01

Understand

The business problem, the users, the workflow around the feature, the constraints, and what success actually looks like.

02

Plan

Architecture, data flow, risks and dependencies agreed before code — the cheapest place to fix a wrong decision.

03

Build

Modular, typed, reviewable implementation that follows the existing conventions of the codebase rather than my personal preferences.

04

Validate

Edge cases, permissions, concurrent access, failure paths and integrations — not just the happy path.

05

Improve

Measure first, then optimise the actual bottleneck. No guess-driven performance work.

TECHNICAL STACK

Tools I use in production

CORE

JavaScript TypeScript React Next.js

FRONTEND ARCHITECTURE

Redux Toolkit TanStack Query React Hook Form

Zod

UI

Tailwind CSS Framer Motion Lucide React SASS

PLATFORMS

Web Applications React Native Electron.js

Progressive Web Apps

BACKEND

Node.js NestJS Express.js Django

DATA

PostgreSQL Prisma MongoDB

INTEGRATION

REST APIs WebSockets Axios SIP.js

FreeSWITCH

AI

OpenAI API AI chat interfaces

LLM integration patterns

LET'S TALK

Have a project that has to **work under load**?

I take on contract and full-time work remotely, with hours arranged to overlap US, UK and Gulf business days. Send over the problem — not a spec — and I will come back with how I would approach it.

Project build

A defined platform or module, scoped and delivered end to end — architecture, implementation, validation and handover.

Team augmentation

Joining an existing engineering team as a senior frontend or full-stack developer, working to your conventions and your process.

Rescue & refactor

A React codebase that has become slow to change, or a real-time feature whose state keeps drifting. Diagnosis first, then a staged fix.

Ongoing retainer

Continuous feature delivery and maintenance on a live product, with a predictable monthly commitment.

EMAIL

gireesh.rajput786@gmail.com

PORTFOLIO

gireeshsingh.vercel.app

LINKEDIN

linkedin.com/in/gireesh-singh-293b83147